

Hands-on session

Splice Site Detection using Convolutional Neural Networks

Date: Nov /18 /2019

Group No.:10.....

Student Name:Ki Seong Lee.....

Student Name:Dongho Lee.....

1. Introduction

Welcome to the hands-on session on Splice Site Detection using Convolutional Neural Networks. In this hands-on session, we will learn how neural networks can be used in practice, and in particular, how neural networks can be used to tackle problems in the field of genomics.

2. Submitting and plagiarism policy

To hand in your solutions for the exercises of this hands-on session, we ask to submit one .zip file per group to the relevant problem set folder of your group on Ufora. This .zip file needs to contain the files listed below, having the content described below:

- **ssd_<group number>.pdf**
- **ssd_<group number>_training.xls**
- **Main.py**
 - The main function that you will be running. Put function calls to the other python files that you create here.
- **MyInputReader.py**
 - At least one function, which takes two data files and returns two lists (X, containing the one-hot vectors, and Y, containing the labels)
- **MyNetwork.py**
 - As many functions as you want, each returning a `NetworkModel` object with the layers you picked.
- **MyMetrics.py**
 - At least three functions, one for each of recall, precision and f1
- **MyFastaPredictor.py**
 - At least one function, that goes through the different steps as described in the last section of this document.

The **deadline** for submitting is **Monday, November 18, 18:00.**

Plagiarism policy: Discussing algorithms is encouraged. Simply copying source code, however, is not allowed: the submitted solutions should be own work, demonstrating a thorough understanding of the problems at hand. As such, upon detection of plagiarism, both groups sharing and copying will get a score of zero.

3. Making use of the GPU server

Deep learning typically requires a huge amount of calculations to be done. If we were to process everything on our laptop, and we want to analyze a big amount of data, calculation times would get unacceptably high. Luckily, we have access to much faster systems. GUGC owns several GPU (graphical processing unit) servers, each yielding a vast amount of computational power. The machine we will be working on has the following specifications:

- Intel Xeon CPU
- 2 GeForce GTX 1080 GPUs with a total of 16 GB of GPU memory
- 16 GB of system memory (RAM)
- 512 GB SSD

Each student will get access to this machine to conduct the experiments that are part of this hands-on session.

Login information

To make use of the GPU servers, you need to go through the following steps:

1. Connecting to the server:

- Open the PuTTY SSH client (or another SSH client), and make use of the following settings:
 - server IP address: 223.194.227.150
 - SSH port number: **222** (!)
 - server account name: see the problem set folder of your group on Ufora
 - server password: see the problem set folder of your group on Ufora

2. Uploading files to the server:

- To upload files to the server from Windows, we suggest making use of an FTP client like FileZilla or WinSCP.
- Log on by filling in the server IP, port number, account name and password.
- Browse to the desired folders on your Windows and your Linux file systems and drag-and-drop the files.

3. Testing your files on the server

- To run python code on the server, you will first need to transfer the python file you wrote in PyCharm / Eclipse. Next, after logging in using ssh (PuTTY), you can run it by typing `python <name>.py`.
- For some parts, it might be easier to first test your code offline with some sample inputs. You can also transfer the data files from the server to your personal laptop if necessary.

- i. The preprocessing part (conversion to one-hot encoding) is something you could easily test offline first.
- ii. The postprocessing part (selecting GTs from the fasta file) is also something you could easily test offline.
- iii. For the network creation and training however, you will only be able to test this on directly on the server.

4. Files already on the server:

In your home directory, you can find a python file, and a directory with the input files for this hands-on session. These files can also be found in the problem set folder of your group on Ufora, should you lose them, or should you wish to download them locally to your PC.

Code (API)

To help you with the creation of your convolutional neural networks, we provide an application programming interface (API) with a class and functions for you to use. This API is built on top of the TensorFlow framework, which is a deep learning framework created by Google, and which makes use of the GPUs of the server for extremely fast calculations.

You should find the file **NetworkModel.py** in your home directory after logging in to the server. Here, you will find the class `NetworkModel`, which you will use. An object of this class represents a neural network, which you can build, print, train, evaluate, save and load. Below, the functions are discussed in detail.

Note that we put an upper limit on the size of the network. It should not have more than 20 layers (which is already a lot for this relatively small dataset).

Note that you are only required to use the functions listed below. Other functions, starting with an `_`, are used internally.

`__init__(self, file_to_load = None)`

The constructor for the `NetworkModel` class comes with one optional argument. If no filename is given in input, it just creates a new, empty neural network model from scratch. If a filename is given, it loads a model that was previously saved to that file using the `saveModel(...)` function. Note that loaded files can only be used for evaluation (`generatePredictions(...)`), and not for (re)training or adding extra layers.

`addInputLayer(self)`

This function adds an input layer to the network. This is the first layer that should be added to every network model.

addConvLayer(self, num_filters, filter_width, zero_padding = True)

This function adds a convolutional layer to the network, with the specified arguments. You should always specify the number of filters (with an upper limit of 500), and the filter width (with an upper limit of 64). There is also an optional argument which lets you choose whether or not zero padding is added (as explained in the lecture slides). After each convolutional layer, a rectified linear unit (ReLU) is automatically added for non-linearization purposes.

addMaxPoolLayer(self, pool_size)

This function adds a max pooling layer to the network, with the specified pool size (with an upper limit of 50).

addFullyConnectedLayer(self, num_of_neurons)

This function adds a fully-connected layer to the network, with the specified number of neurons (with an upper limit of 1000). If it is the first fully-connected layer in the network, it will also add a flatten layer first, which reduces the dimensionality after the convolutional/pooling layers. For instance, if the output of the last pooling layer is $(?, 20, 100)$, the flatten layer will change this to $(?, 2000)$. It is also no longer possible to add convolutional or pooling layers after this. After each fully-connected layer, a rectified linear unit (ReLU) is automatically added for non-linearization purposes.

addOutputLayer(self)

This function adds an output layer to the network, which has two neurons: one for negative classification and one for positive classification. A softmax calculation is also done, so that the probabilities that are outputted by both neurons add up to 1. If no fully-connected layers have been added in the network, it will also add a flatten layer first. Once an output layer is added, no other layers can be added anymore.

printDetails(self)

You can use this function for printing out an overview of the layers that you have added to the network. It can be not only be used for a completed network, but also intermediately, for a network that is still being built. The output is printed out, and will consist of three columns: the type of layer, the hyperparameters (such as the number of neurons) and the output size after this layer. This output size will be shown in a format $(?, 200, 100)$. The question mark indicates the amount of input samples (which is not fixed), so in building your network you can ignore this. In addition, the total amount of trainable parameters in the network topology is also printed.

train(self, trainX, trainY, validX, validY, n_epochs)

To train the network, you will first need to read in the datasets, and convert the sequences to the right format. More in this format will follow in the assignments. After supplying the training and validation sets, you need to specify for how many epochs you want to train (maximum 100). The training and validation losses and accuracies will be shown on the screen. Following the early stopping strategy, the resulting model is automatically chosen as the model with the lowest validation cost over the different epochs.

generatePredictions(self, testX)

This is the function you will use to generate predictions for a certain dataset. In input, you give a list of sequences, in the one-hot encoding format (again, see the assignments). It will output a list of probabilities as calculated by the network. The list is a two-dimensional list of size $(n, 2)$, with n being the number of sequences in `testX`. This means that the list will be a list will contain n lists of size 2, each containing the negative and positive probability for the prediction.

Note that the order of the results is the same as the order of the sequences in input.

saveModel(self, file_to_save_to)

At any time, you can save your current model (with its layers and trained parameters) to a file. However, it really only makes sense to do this after training. To load this model again, create a new `NetworkModel` object, and specify the same filename in the `file_to_load` parameter. Note: after loading a model, it cannot be (re)trained.

We recommend to use this when you are working on the evaluation. Train a model once, save it, and then test the remainder of your code by loading that model.

Data

In the directory *inputs*, we can find the datasets for this hands-on session. These contain the sequences that we will process. All sequences are extracted from the genome of the Arabidopsis Thaliana plant. Have a look at the file contents to see the format in which the sequences are stored.

We provide a training set, a validation set and a test set. Each of these sets consists of a file with all positive sequences, and one with all negative sequences. We also added one fasta file, which we will use for our final evaluation.

The filenames you should find are:

- *train.pos* and *train.neg*
- *valid.pos* and *valid.neg*
- *test.pos* and *test.neg*
- *full_sample.fasta*

4. Preprocessing (8 points)

As a first step in our complete workflow for deep learning, we have to preprocess the data before we feed it to our network. We know that we have a number of sequences stored in the datasets on the server, in some format.

The first step is to have a look at the content of the files on the server and ask yourself: “How do the data look like? What is the data format? Do we need all information present in the file?”

Q1: Are we dealing with datasets containing information about donor sites or acceptor sites?

we are dealing with datasets containing information about donor sites.

Q2: How many positive and negative samples do we have in our train set, validation set and test set separately?

Number of positive samples in our train set is 30000 and number of negative samples in our train set is 6000. So, number of training examples are 36000.

Number of positive samples in our validation set is 7500 and number of negative samples in our validation set is 1500. And the total number of validation examples are 9000.

Number of positive samples in our test set is 7500 and number of negative samples in our test set is 1500. Total number of test examples are 9000

We now know what our data look like, so we can convert the data to the format that our network expects. In the slides, we discussed how to represent our DNA sequences as numerical vectors.

Q3: Could you think of a reason why we prefer this one-hot representation over for instance a binary representation, where we would actually need only half as much storage? ($A = [0\ 0]$, $C = [0\ 1]$, $G = [1\ 0]$, $T = [1\ 1]$)

Using one hot representation ensures that only identical nucleotide contributes to neuron activation. Using binary representation may result in overlap and ambiguity in neuron activation, because numbers in the index of binary representation may overlap with the other nucleotide. For example, adenosine has 0 in index 0 and so does cytosine. This may result in undesired activation of neuron.

P1: Create a Python script `MyInputReader.py` that defines one or more functions for reading the input datasets and converting them to a one-hot representation. We advise to make a function that takes the names of two data files as input (for instance, 'train.pos' and 'train.neg'), and which returns the trainX and trainY lists as discussed below. We also advise to create a separate function for the conversion of one sequence to its one-hot encoding, as you will need to do this in Section 8 as well.

Create a file `Main.py` that uses the functions you defined in `MyInputReader.py`. This file will also be used to call functions from the other files that you will create. Basically, it should use the code of all different files, combining the different functionalities, in a clean and structured way.

The format to which you should convert the sequences is as follows:

As each sequence in the provided files has an exact length of 200, each sequence should be converted to a list of exact 200 vectors, namely $[1, 0, 0, 0]$ for A, and so on. If we have m positive and n negative sequences, the dimensions of trainX should be $(n+m) \times 200 \times 4$. Our trainY should be of dimension $(n+m)$.

An example with two small files:

`sequences.pos`

ACG

`sequences.neg`

TAC

GAT

Would give us:

```
trainX = [ [[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0]] , [[0, 0, 0, 1], [1, 0, 0, 0], [0, 1, 0, 0]] , [[0, 0, 1, 0], [1, 0, 0, 0], [0, 0, 0, 1]] ]
trainY = [1, 0, 0]
```

Note: it is not necessary to shuffle the positive and negative examples; this is already done during each epoch at training time.

5. The network topology (6 points)

P2: After preprocessing the input data to a usable format, it is time to build our neural network. To do so, create a python script `MyNetwork.py`, where you write one or more functions that return a new `NetworkModel` object. Please follow the guidelines of the API, as given in the Code section earlier in this document. We give you (almost) complete freedom in constructing your own network topology, as there are no fixed rules on how this should be done.

We ask you to create at least three network topologies:

- A regular neural network, with an input layer, two fully-connected layers with each 50 neurons, and an output layer. Note that no convolutional or pooling layers should be added here.
- The basic convolutional neural network that was shown in the slides (slide number 18)
- Your own convolutional neural network. For this topology, you can start from the basic network (b) and add more layers, filters, neurons, etc.

With your own network, try to improve the validation cost of the basic network as much as possible. Further comparisons (using other metrics) will be made in Section 7.

6. Training (8 points)

P3: After creating a network topology, you can start training it, by calling the `train(...)` function of the `NetworkModel` object. This will update the parameters using the backpropagation algorithm, and at the end of each epoch, show the cost and accuracy.

After trying a couple of different topologies (and comparing the best validation cost for each), create an Excel (or similar) file `ssd_<group_number>_trainings.xls`, in which you store the costs and accuracies obtained for each epoch (for both the training and validation set) for the three different network topologies, and show the progress in a graph. Hint: make use of a special paste in Excel, using `'|'` as a delimiter.

Remember: If desirable, you can save your `NetworkModel` object after training by using the `save(...)` function. To load it again later, give the same filename to the `__init__(...)` function when creating a new `NetworkModel` object.

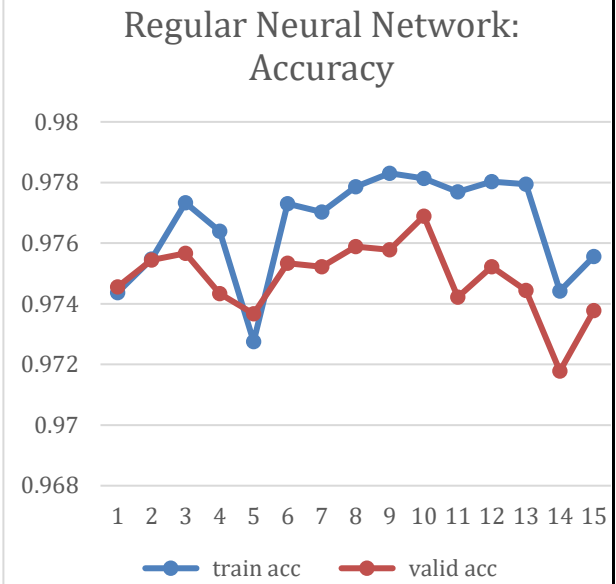
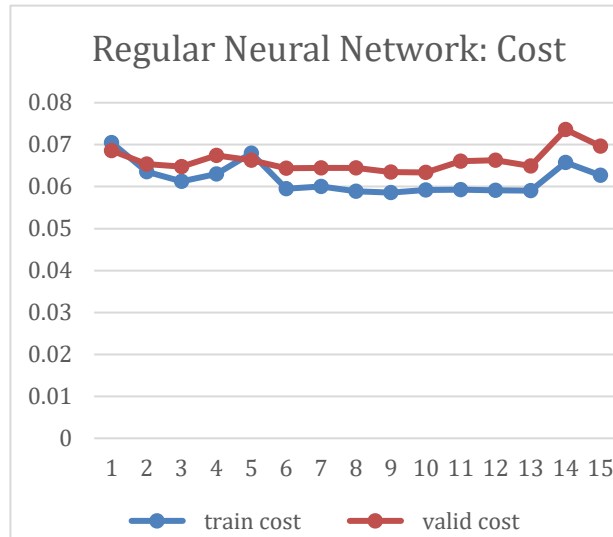
Interesting: To investigate the GPU usage on the server while training, you can make use of the unix command `nvidia-smi`. You can see the temperatures of each GPU, their memory usage, performance level, etc. Have a look what happens while training (in a different PuTTY ssh session), or when your colleagues are using them.

Q4: As we did not specify a metric yet, the default metric, accuracy, is used. Answer the following questions by looking at the training and validation costs and accuracies while training the three models.

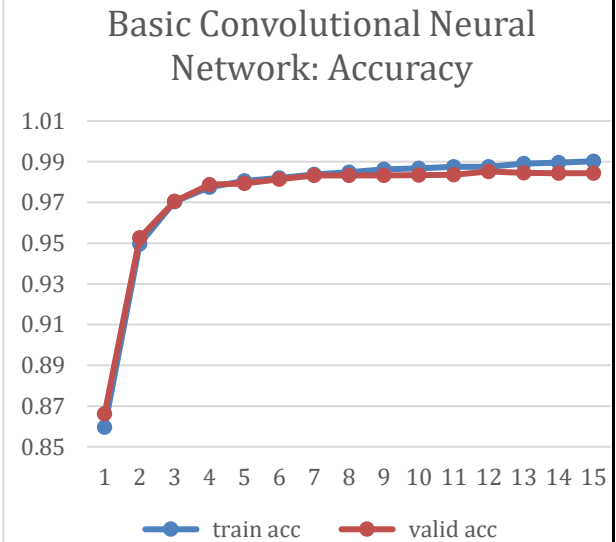
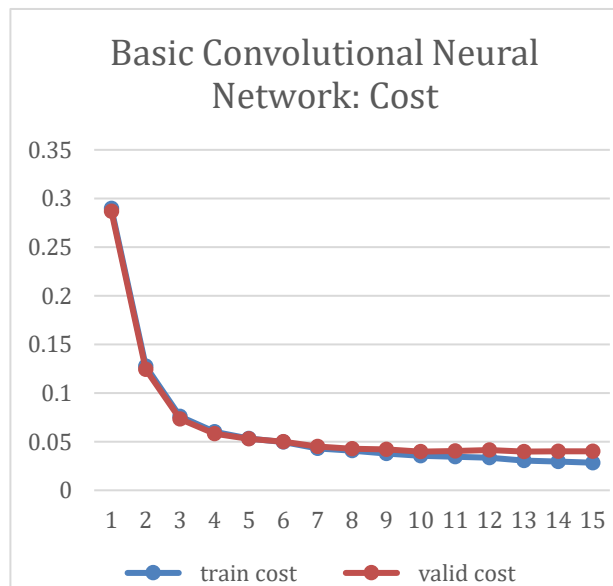
- *Is there a difference in training cost between the different models? Are there models that achieve a better training cost than the others? Can you explain this?*
- *How about the difference between training cost and validation cost within one model? Is this difference similar for the different models, or is there a model where the gap between training cost and validation cost is bigger?*
- *Does your model achieve a good fit at the end of training? Explain, by using the training and the validation results.*
- *In the lecture slides, we talked about the difference between the behavior of the training cost and validation cost, and the early stopping algorithm. When comparing the training and validation costs during training for your own network, do you notice the same behavior? If you do not see it at first, try increasing the number of epochs. Also, as stated before, the API automatically applies the early stopping algorithm, so the final model after training will be the one at the epoch with the lowest validation cost.*

Training cost differed among the models. Some models achieved lower training cost with better accuracy. The cost function represents how much the training output strayed away from the expected output fed to the network. Some model had higher discrepancy between validation and training cost. The model did not achieve a good fit at the end of the training, because validation cost was much higher (40 times the training cost) than training cost throughout the epochs, hinting it is an overfitting model. Increasing epochs actually made the discrepancy between the training and validation loss to increase, where training loss kept going down while validation increasing slightly.

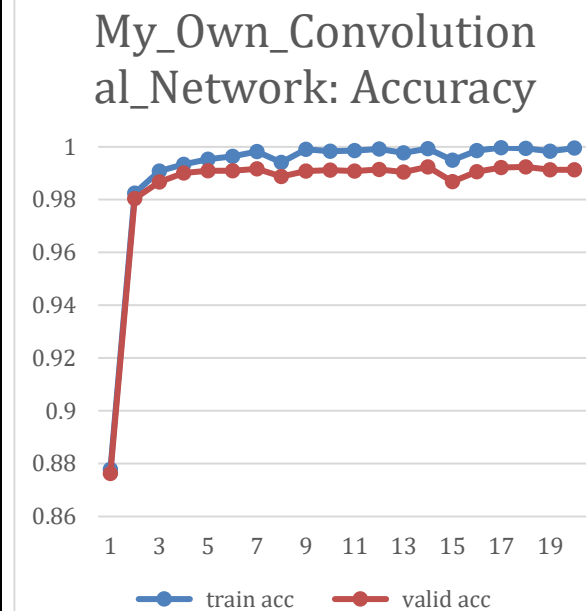
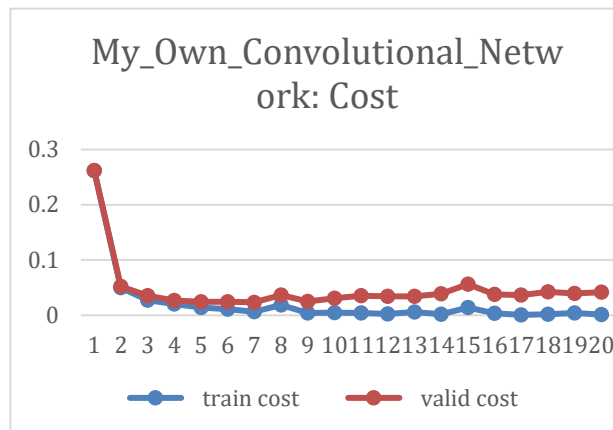
(a) Regular neural network



(b) Basic convolutional neural network



(c) Your own convolutional neural network



Network information:

This network has 238582 trainable parameters.

0. Input layer		-> Output size: (?, 200, 4)
1. Convolutional layer	10 filters, width 5, zero padding, with ReLU	-> Output size: (?, 200, 10)
2. Convolutional layer	30 filters, width 8, zero padding, with ReLU	-> Output size: (?, 200, 30)
3. Max pooling layer	pool size 2	-> Output size: (?, 100, 30)
4. Convolutional layer	50 filters, width 10, zero padding, with ReLU	-> Output size: (?, 100, 50)
5. Convolutional layer	30 filters, width 20, zero padding, with ReLU	-> Output size: (?, 100, 30)
6. Max pooling layer	pool size 2	-> Output size: (?, 50, 30)
7. Convolutional layer	30 filters, width 20, zero padding, with ReLU	-> Output size: (?, 50, 30)
8. Convolutional layer	30 filters, width 25, zero padding, with ReLU	-> Output size: (?, 50, 30)
9. Flatten layer		-> Output size: (?, 1500)
10. Fully-connected layer	100 neurons, with ReLU	-> Output size: (?, 100)
11. Softmax (output) layer	2 neurons	-> Output size: (?, 2)

7. Evaluation (10 points)

In the slides, we suggested the use of two metrics to evaluate imbalanced datasets: precision and recall. Let us have a more detailed look at these two metrics.

Q5: Given the following predictions (accompanied by their actual classes), what is the accuracy, recall and precision of our two models? Compare the three scores of each and discuss. Are the metrics relevant in our biological context, where we do not want to miss out on too many splice sites? Which model is preferable in this context?

Seq #	Actual class		Model 1 pred	TP/FP/TN/FN ?		Model 2 pred	TP/FP/TN/FN ?
1	1		0.35	FN		0.67	TP
2	1		0.55	TP		0.88	TP
3	1		0.88	TP		0.53	TP
4	1		0.45	FN		0.65	TP
5	1		0.44	FN		0.99	TP
6	1		0.12	FN		0.47	FN
7	1		0.38	FN		0.94	TP
8	1		0.22	FN		0.89	TP
9	0		0.22	TN		0.02	TN
10	0		0.35	TN		0.03	TN
11	0		0.11	TN		0.16	TN
12	0		0.04	TN		0.20	TN
13	0		0.07	TN		0.31	TN
14	0		0.12	TN		0.53	FP
15	0		0.04	TN		0.89	FP
16	0		0.22	TN		0.26	TN
17	0		0.49	TN		0.52	FP
18	0		0.47	TN		0.66	FP
19	0		0.35	TN		0.73	FP
20	0		0.12	TN		0.59	FP
21	0		0.15	TN		0.03	TN
22	0		0.23	TN		0.61	FP
23	0		0.48	TN		0.38	TN
24	0		0.15	TN		0.71	FP
25	0		0.16	TN		0.81	FP

Model 1:

- Accuracy = 76%
- Recall = 0.25
- Precision = 1

Model 2:

- Accuracy = 60%
- Recall = 0.875
- Precision = 0.4375

The metrics may be relevant to biological context depending on primary objective of the model. If the model is aimed to identify as much donor sites as possible. In this context, it is important to minimize false negative, as false negatives are true donor sites that are recognized as acceptor sites. Due to this reasoning it is better use model 2, which has higher recall and lower number of false negatives.

As you can tell, it is not easy to distinctively compare the two models. Is a model with high precision and low recall better than a model with low precision but high recall, or vice versa? This is dependent on the use case, and often we want to find some middle ground. If we want a general performance measure of our model, we must find a way to combine precision and recall into one metric. This is exactly what the F1 score tries to achieve.

The F1 score is the harmonic mean of precision and recall.

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Calculate the F1 score for both and compare again. Are both systems comparable now, or is there a clear winner?

Model 1, F1 score: 0.4

Model 2, F1 score: 0.5833

Model 2 had higher F1 score than model 1, however they are still low overall. There is no clear winner in this case.

P4: After training our model, we want to evaluate its performance on an independent test set. To do so, create a new file MyMetrics.py, where you define three functions to calculate the recall, precision and F1 score. Take the following steps to execute them (in your Main.py file):

1. Read the test.pos and test.neg dataset by (re)using the code using your MyInputReader.py file.
2. Load your previously trained model (or train a new one).

3. Generate predictions on the test set by using the `generatePredictions(...)` function of the `NetworkModel` object.
4. Use the code in your `MyMetrics.py` file to calculate the recall, precision and F1 score for the predictions of the previous step. In addition, make the precision and recall functions also print out the number of TPs, FPs, TNs and FNs.

If all went well, you should see an improvement of performance for networks (b) and (c). For the test set, with the right network topology you should be able to achieve an F1 score of around 0.97-0.98 or even higher. If you did not reach this score, try modifying your topology by adding/removing/changing layers. Try some different topologies to achieve the highest possible score, and feel free to compare your scores with your fellow students!

It is very interesting to note that the amount of trainable parameters of topology (a) is more than 12 times greater than the amount of trainable parameters of topology (b), yet you will clearly see that it performs worse because it lacks convolutional layers which are very suited for this task, as explained in the slides. Long live the power of CNNs!

(a) Regular neural network

Amount of trainable parameters: 42702

TP : 1394 FP : 120 FN : 106 TN : 7380

recall = 0.9293333333333333

precision = 0.9207397622192867

f1 Score = 0.9250165892501658

(b) Basic convolutional neural network

Amount of trainable parameters: 3757 trainable parameters.

TP: 1429 FP: 69 FN: 71 TN: 7431

Recall = 0.9526666666666667

Precision = 0.9539385847797063

F1 score = 0.953302201467645

(c) Your own convolutional neural network

Amount of trainable parameters:

TP: 1467 FP: 37 TN: 33 FN: 7463

Recall: 0.978

Precision: 0.975398936170218

F1 score: 0.9766977363515312

8. Using a trained model in practice (8 points)

P5: Now that we have a trained model and we verified its performance on an independent test set, we can put this model to use. In the file `full_sample.fasta` that you find amongst the data files on the server, we have a long DNA sequence in which we want to find the donor (GT) splice sites. To do so, create a python file `MyFastaPredictor.py` and create a function that follows the following steps:

1. Read the fasta file. This file contains one long DNA sequence, but it is divided into multiple lines. Concatenate the different lines to get to one string that has the full DNA sequence.
2. Next, extract all candidate donor sites. We only work with canonical splice sites, so you should only extract the GT pairs. Also, you should only extract the GT pairs with sufficient context around them. As our training set has sequences of 200 nucleotides ($99 + 2 + 99$), you should only select candidate splice sites from index 99 onwards, and until 99 nucleotides from the end (as otherwise the candidate sites would not have enough context for our model).
(Hint: the total amount of candidate splice sites in the fasta file should be 224)
3. Convert the list of sequences to generate the one-hot encodings (try to reuse code in `MyInputReader.py` for this!).
4. Load your previously trained model (or train a new one).
5. Predict the (positive) probability for each candidate splice site.
6. Print out your results in a clean output format, where you list the candidate splice site's position, the local context (± 10 -15 nucleotides) and the predicted probability in different columns.
As an example of the information you should enlist, here is a screenshot of the online prediction tool that was shown in the theory lecture:

Position	Hit (with context)	Score
200	TCGAGGTAAGTG	0.258857
204	GGTAAGTGTTAT	0.002707
206	TAAGTGTTATTT	0.031683

113		GAATGCAAGCGTATCGTCTCCG		1.7661264450907765e-07
118		CAAGCGTATCGTCTCCGAGAGC		4.584197910162402e-08
133		CGAGAGCGAGGTCCTTTTCTC		0.9997996687889099
171		CTTCTCTCTTGTTAATTGTCCG		0.010248210281133652
178		CTTGTTAATTGTCCGCGTTTTG		0.014358109794557095
184		AATTGTCCGCGTTTTGGTCAAT		0.0003454843536019325
190		CCGCGTTTTGGTCAATCGACGC		0.0019632659386843443
219		GCCAATTAGGGTTTTAGTGATT		1.4511933841276914e-06
225		TAGGGTTTTAGTGATTTTGATA		1.919793390925406e-08
243		GATATACTTCGTACGAGAAGGG		4.031782907532033e-08
254		TACGAGAAGGGTATGCGTGTGG		0.00016119881183840334
260		AAGGGTATGCGTGTGGCTGTCT		6.524349771552806e-08
262		GGGTATGCGTGTGGCTGTCTGT		6.177295119869086e-08
268		GCGTGTGGCTGTCTGTGGCACA		6.624907200603047e-08
272		GTGGCTGTCTGTGGCACAGTTG		6.876434355262973e-08
280		CTGTGGCACAGTTGGCTCTGGA		0.00014844928227830678
299		GGAAAATCAAGTTTTATCTCTT		0.0013839646708220243
346		CTCTGGCGAAGTAAGTTAATGT		0.9988318085670471
350		GGCGAAGTAAGTTAATGTTTTG		3.539192405810354e-08
356		GTAAGTTAATGTTTTGGGGTGT		3.5877802950778914e-09

